PukSquash Developer Manual

Administrator User Management

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

User management lets an ACTIVE ADMIN inspect application users, change role or status, and delete an account. The application role is held in Data Connect rather than inferred from a browser flag. Account deletion touches both Firebase Authentication and the relational profile.

ManageUsers loads the admin user list and submits selected changes. The routes run FirebaseAuth followed by AdminMiddleware. The controller validates allowed role and status values and delegates to UserRepository. Deletion first removes the Firebase Authentication account and then removes the Data Connect User record.

1 1 Files and Responsibilities

resources/js/pages/ManageUsersPage.vue — user list and edit actions

app/Http/Middleware/AdminMiddleware.php — ACTIVE ADMIN requirement

app/Http/Controllers/Api/AdminUserController.php — list, show, update and destroy

app/Repositories/UserRepository.php — Data Connect user operations

app/Services/FirebaseAuthService.php — Firebase account deletion

1 2 API Contract

GET /api/admin/users and GET /api/admin/users/{id} read users. PUT /api/admin/users/{id} accepts role and/or status. DELETE /api/admin/users/{id} removes the account. Allowed roles: USER, COACH, ADMIN. Allowed statuses: ACTIVE, INACTIVE, SUSPENDED.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Validating an administrative update

app/Http/Controllers/Api/AdminUserController.php | source line 39 onward; blank and comment lines omitted

```
public function update(Request $request, string $id): JsonResponse
{
    $validated = $request->validate([
        'role'          => 'sometimes|in:USER,ADMIN,COACH',
        'status'        => 'sometimes|in:ACTIVE,INACTIVE,SUSPENDED',
    ]);
    if (! array_key_exists('role', $validated) && ! array_key_exists('status',
$validated)) {
        return response()->json(['message' => 'No changes provided.'], 422);
    }
    $this->users->updateUser($id, $validated);
    $user = $this->users->findById($id);
    return response()->json($user?->toArray());
}
```

Explanation

The update endpoint permits role and status changes but rejects an empty change request. It passes validated fields to the repository and returns the reread profile. AdminMiddleware must run before this method; the controller itself is not the complete access boundary.

Physical Code Explanation

1 sometimes allows changing one permitted field without sending the other. Each value is constrained to its known set.

2 The empty-change branch returns HTTP 422 instead of submitting a no-op update.

3 updateUser sends the update to Data Connect and invalidates cached user data.

4 findById retrieves the updated record for the response so the table can display server-authoritative values.

The baseline has no explicit last-administrator or self-demotion safeguard. Add an agreed policy and tests before treating such changes as operationally safe.

3 Updating Data Connect user fields

app/Repositories/UserRepository.php | source line 104 onward; blank and comment lines omitted

```
public function updateUser(string $id, array $params): void
{
    $existing = $this->findById($id);
    if ($existing) {
        $this->forgetCachedUser($existing);
    }
    $this->client->execute('UpdateUser', array_merge(
        ['id' => $id],
        $params,
    ));
    if ($existing) {
        $this->forgetCachedUser($existing);
    }
}
```

Explanation

The repository clears the current profile's caches before and after UpdateUser. The GraphQL mutation targets the application user ID and writes role/status. Clearing both the ID and Firebase UID caches prevents old privileges being retained merely because a cached profile was returned.

Physical Code Explanation

1 findById obtains the existing identity needed to invalidate both cache keys.

2 array_merge adds the target id to the validated update fields.

3 execute invokes UpdateUser using the server Data Connect transport.

4 The second invalidation prevents a stale lookup made during the update sequence from remaining cached.

destroy removes the Firebase user first; an already missing Firebase identity is tolerated. The subsequent relational deletion is separate. A failed relational delete can leave a profile whose Firebase identity is gone, so account deletion needs a recoverable operational procedure.

4 Verification and Maintenance

Verification checklist

1 Attempt every admin user route as USER and COACH: expect 403. Test an inactive ADMIN as well.

2 Change USER to COACH and refresh the account in the affected session. Confirm coach APIs check the updated server role.

3 Send an unknown role, unknown status or empty update: expect validation failure.

4 Delete a disposable account without related bookings and verify both identity and profile removal.

5 Test deletion with existing booking references and a simulated Firebase or Data Connect failure. Record and repair partial outcomes; do not use real accounts for this test.

Implementation boundaries

The global Firebase middleware in the baseline does not enforce application status for all ordinary endpoints. Changing status is persisted, but complete suspension enforcement requires a shared backend check.

The baseline deletion path does not first resolve booking references or provide a cross-service transaction. Prefer controlled deactivation where history must be retained and define the referential policy.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).